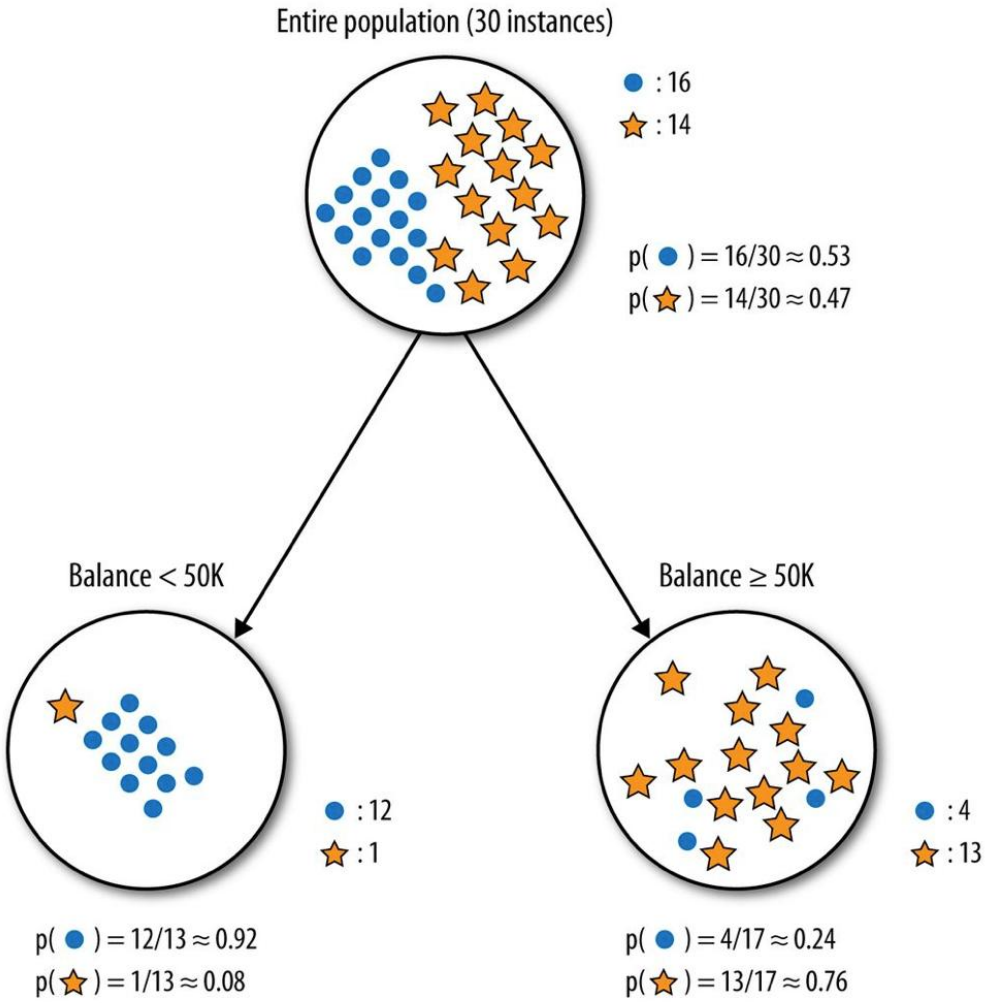
Ensemble methods can use overfitting and underfitting to their advantage

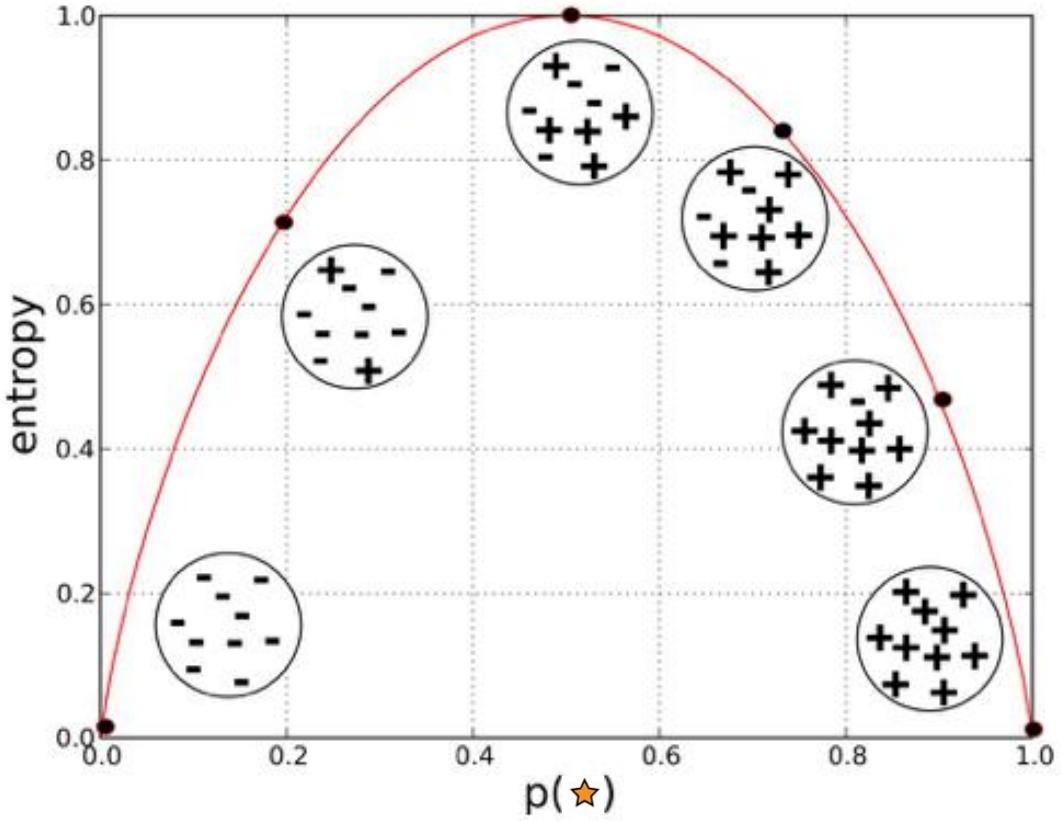
Decision trees serve as the foundation for **ensemble methods** like Random Forest and Gradient Boosting, which combine multiple trees to improve accuracy and robustness.

Ensemble learning reduces **variance and bias**, as bagging (e.g., Random Forest) decreases overfitting, while boosting (e.g., AdaBoost, XGBoost) enhances weak learners by correcting misclassified instances iteratively.

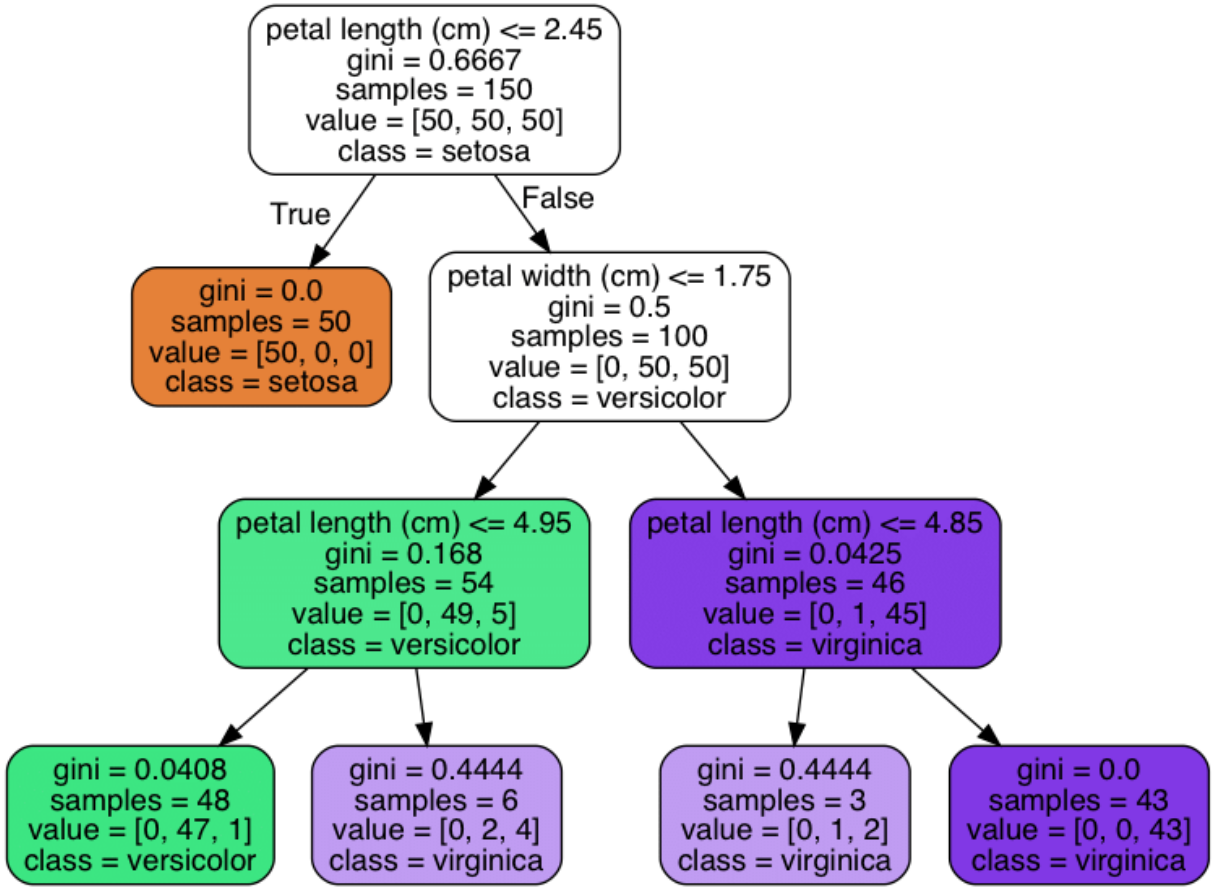
Decision tree models - recap



$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$



Single decision trees have several advantages and disadvantages



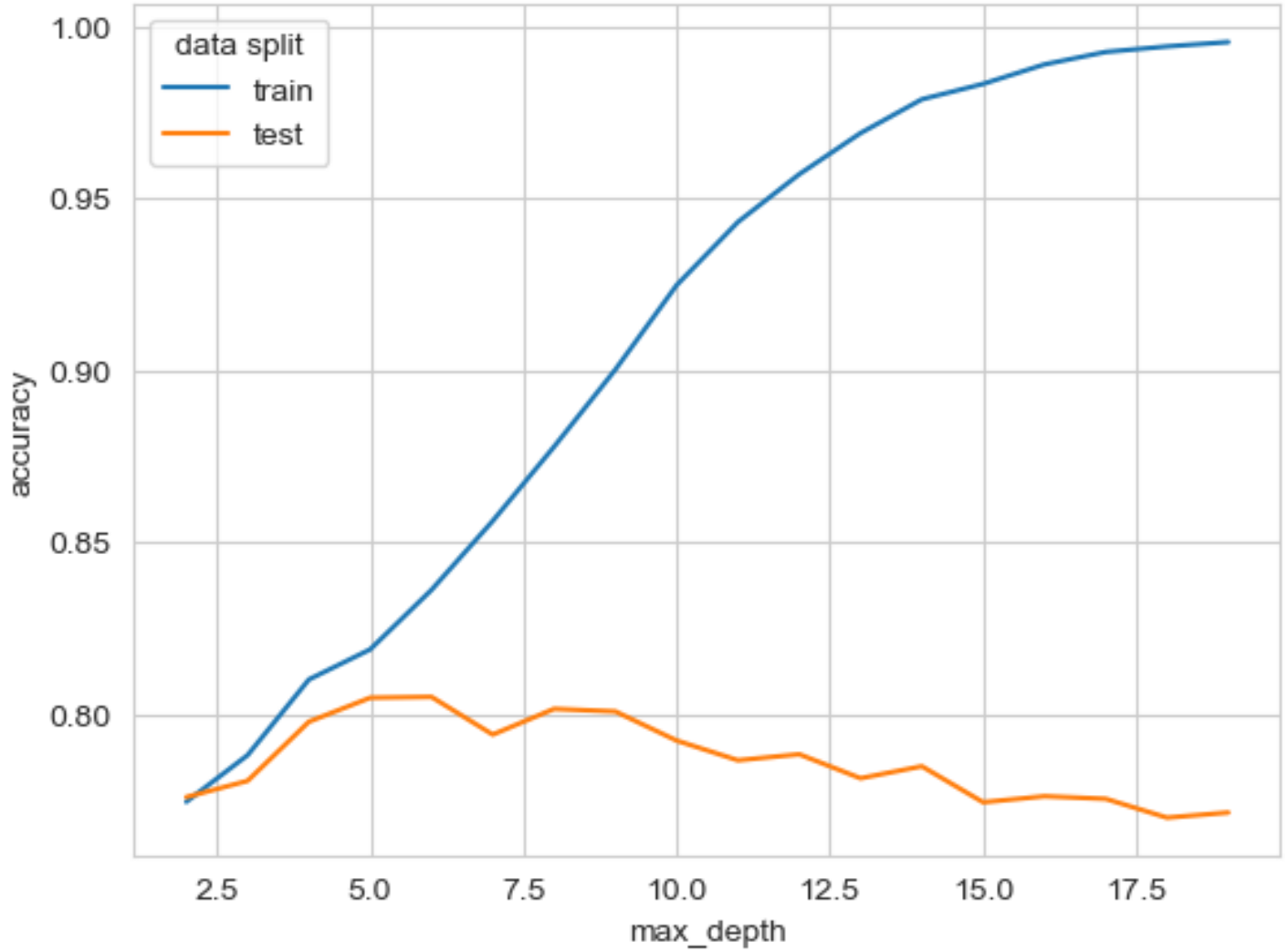
advantages:

ease of interpretation
handles continuous and discrete features
invariant to monotone transformation of features
variable selection automated
low bias (deep trees)

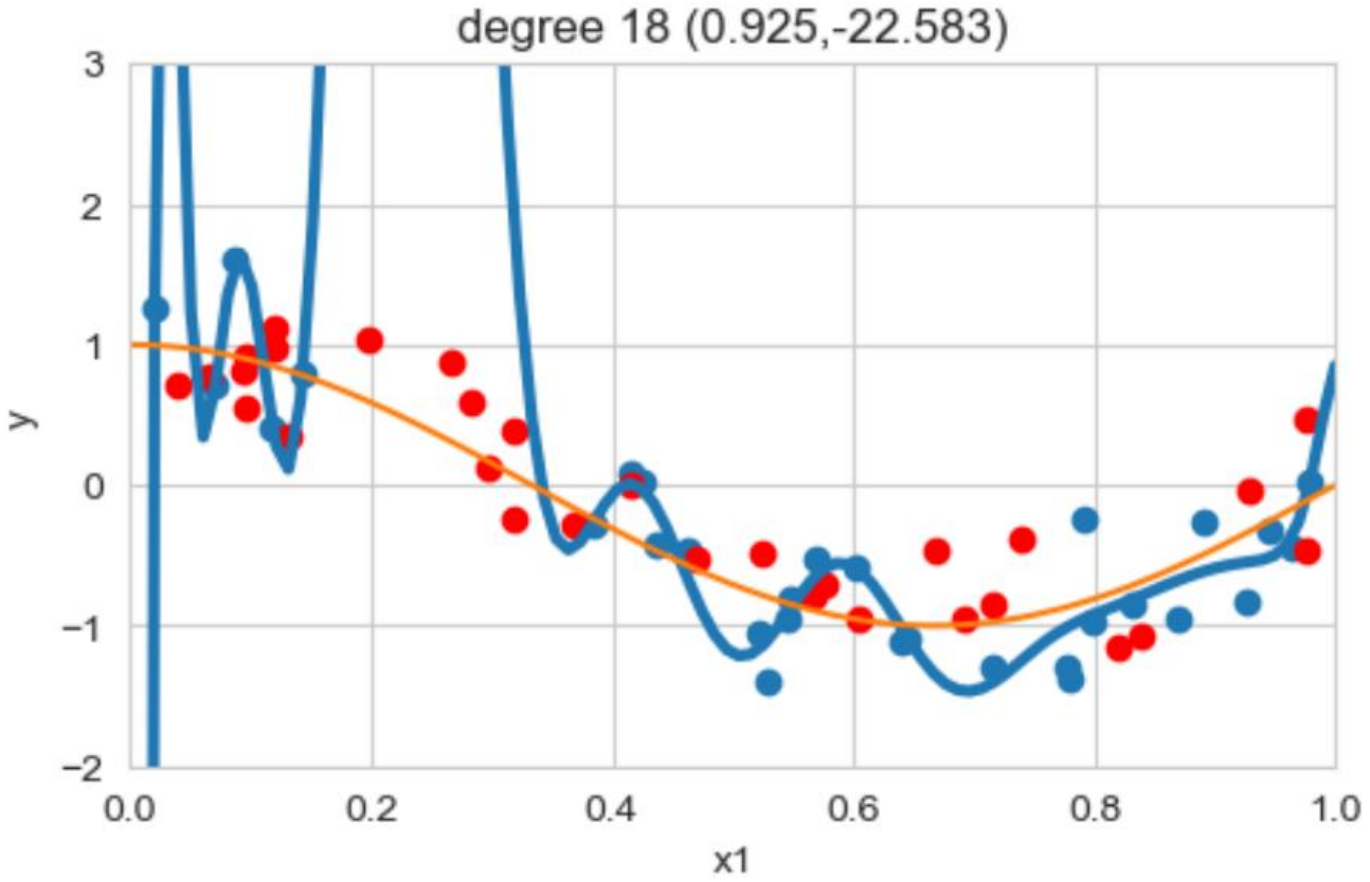
disadvantages:

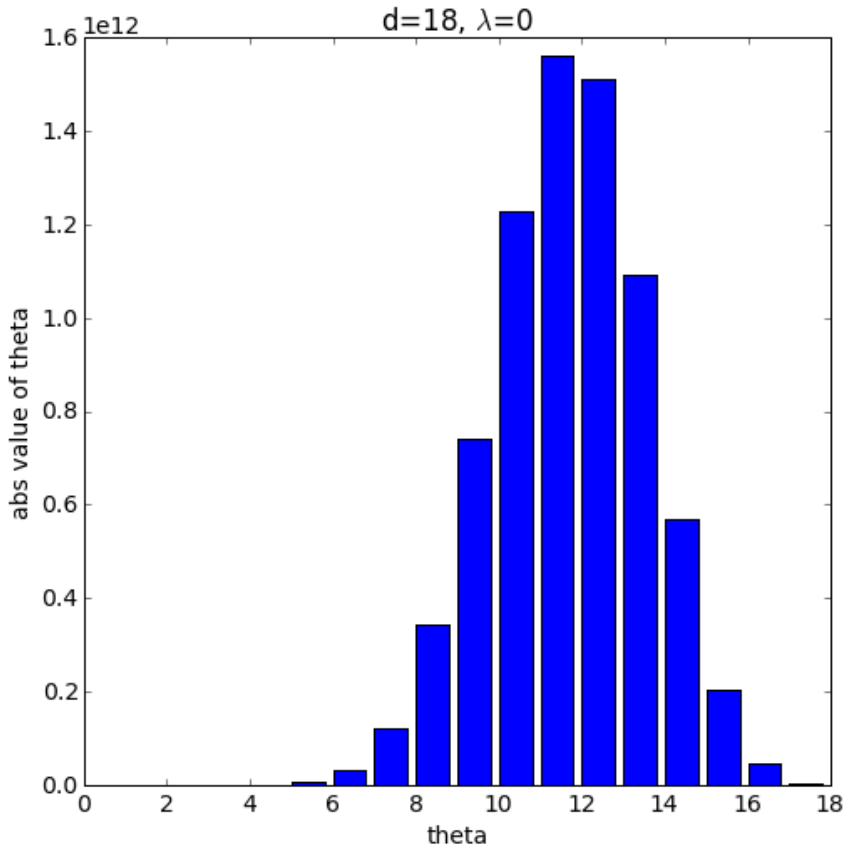
high variance
overfitting

Model hyperparameter optimization on unseen data

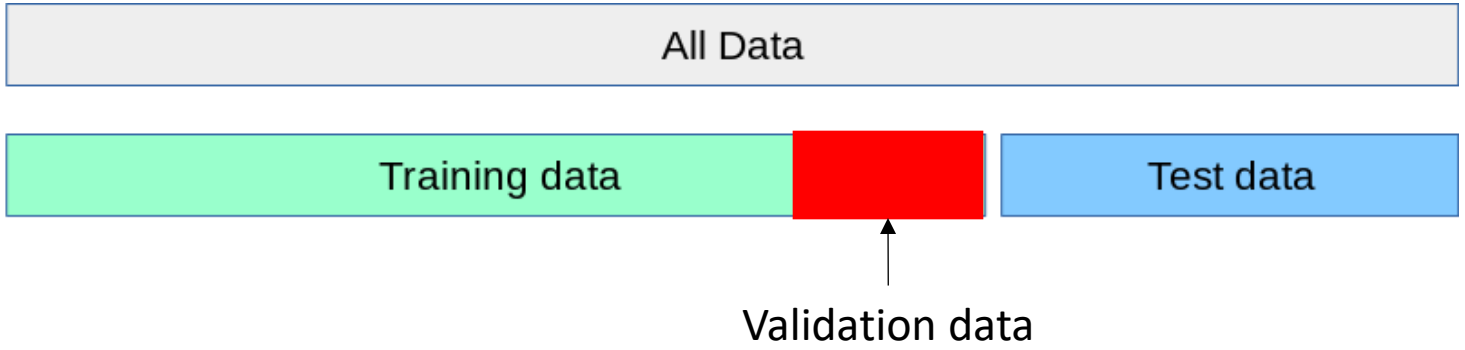


Regularized regression

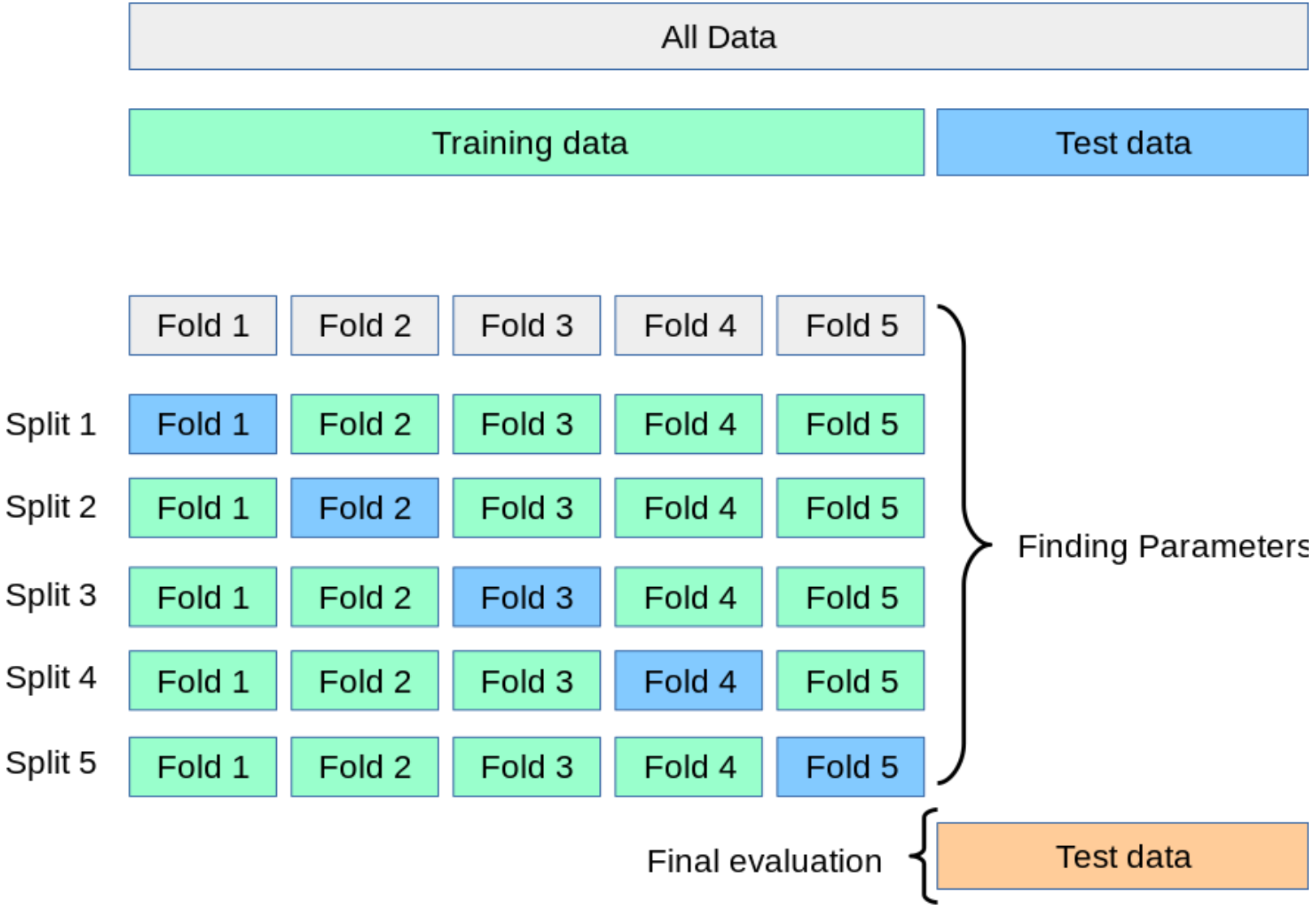




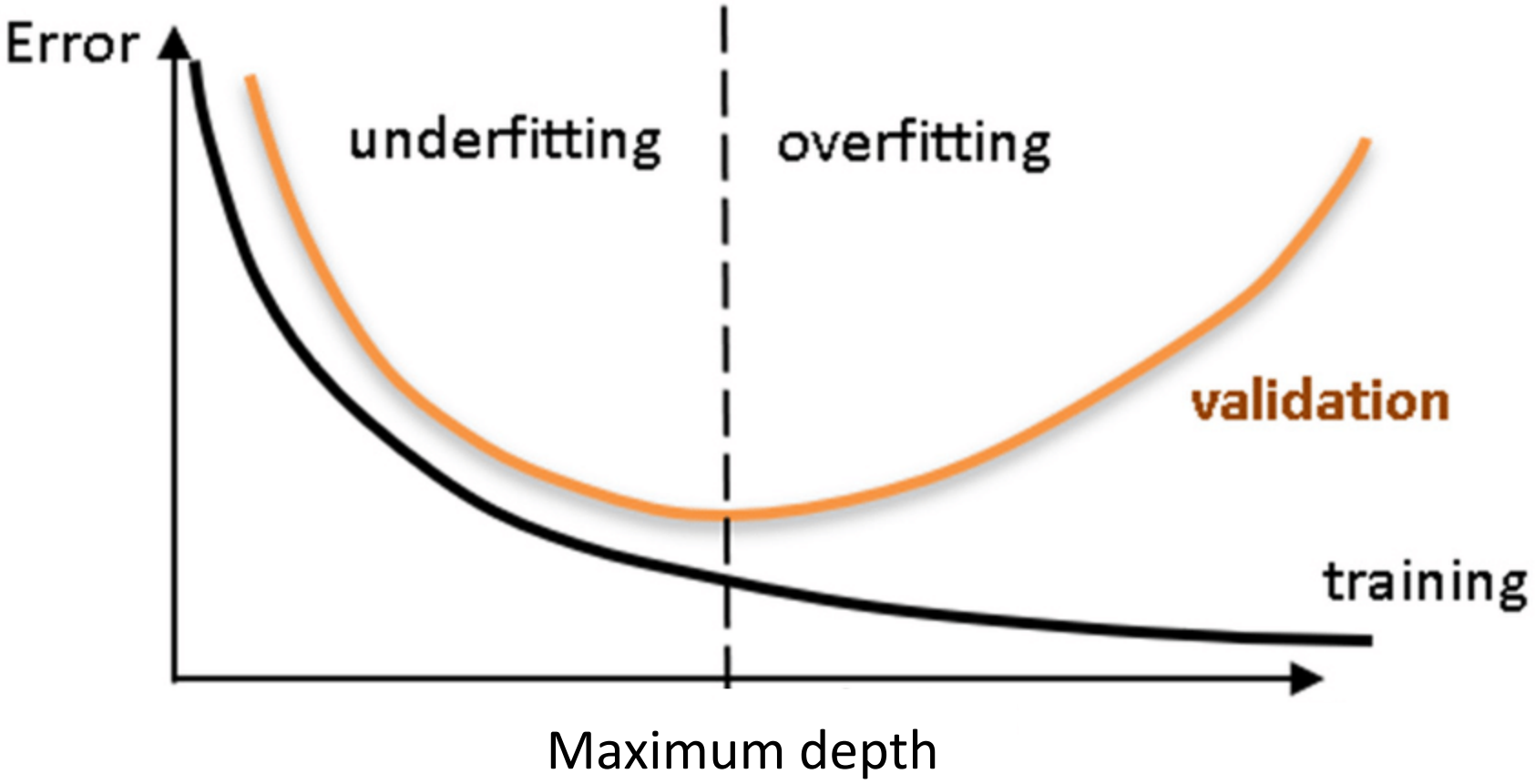
Validation hold-out set



5-fold cross-validation; mostly when not sufficient data is available



Where do we want to sit in terms of underfitting and overfitting

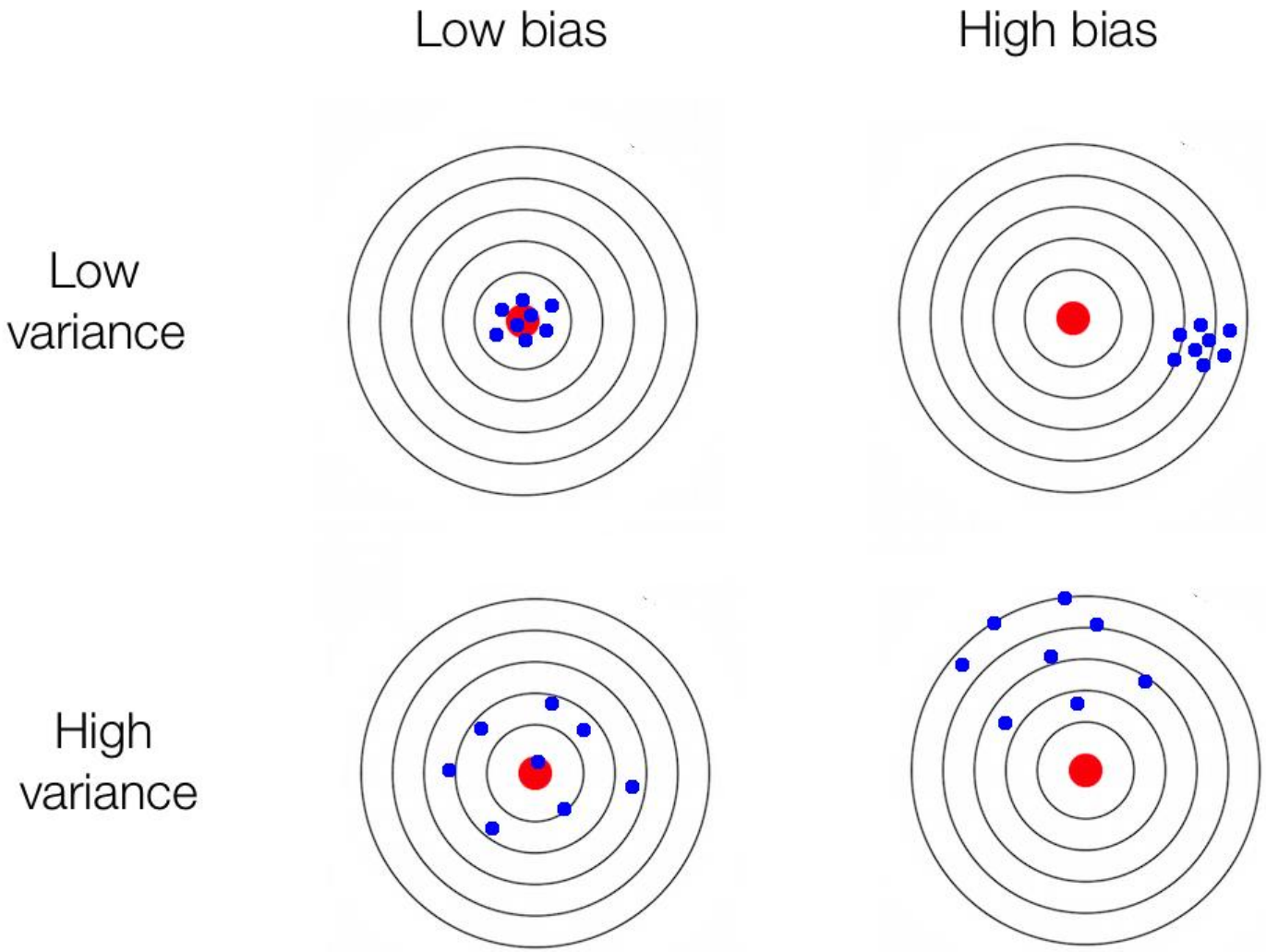


Concept of bias and variance is a foundation of fitting models

Error due to bias (systematic errors, oversimplification)

Error due to variance (sensitivity due to flexibility, overcomplification)

Irreducible error

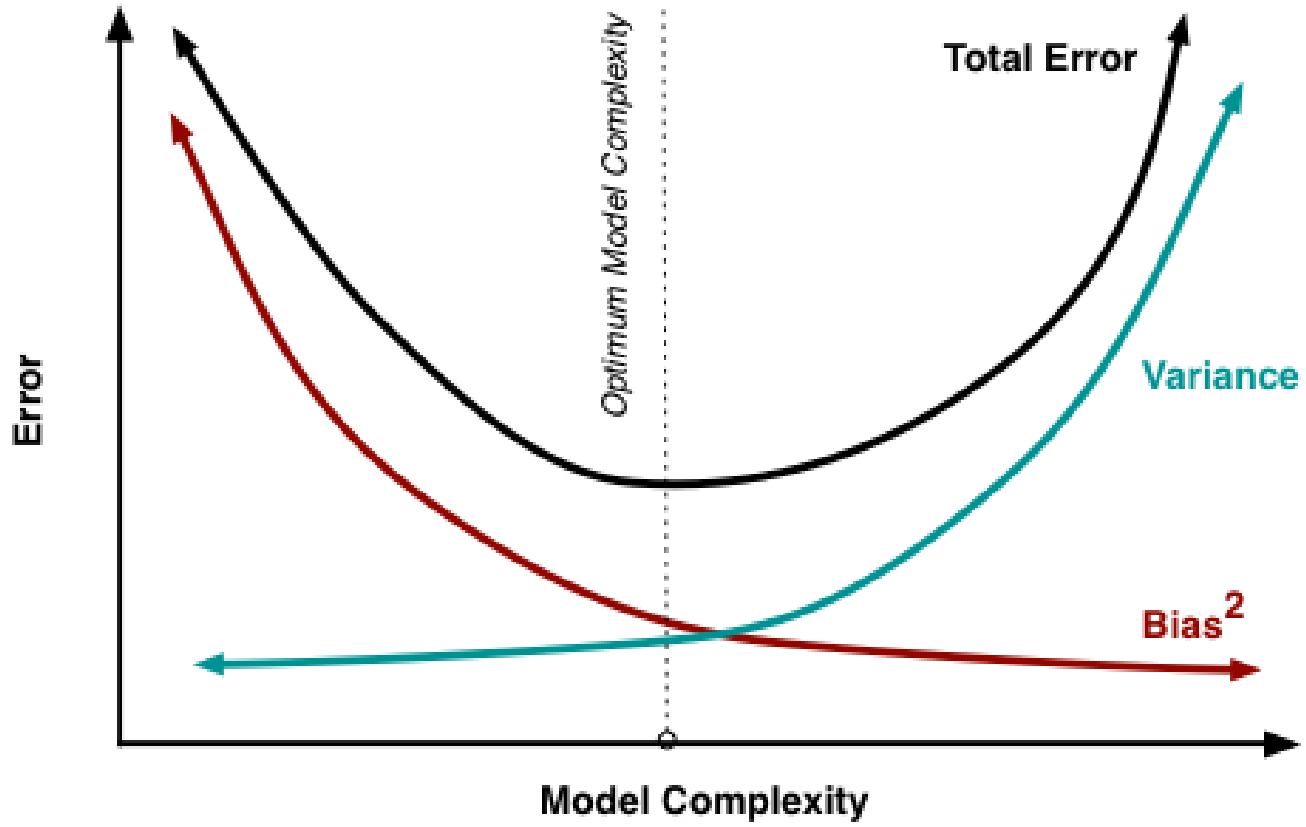


Concept of bias and variance is a foundation of fitting models

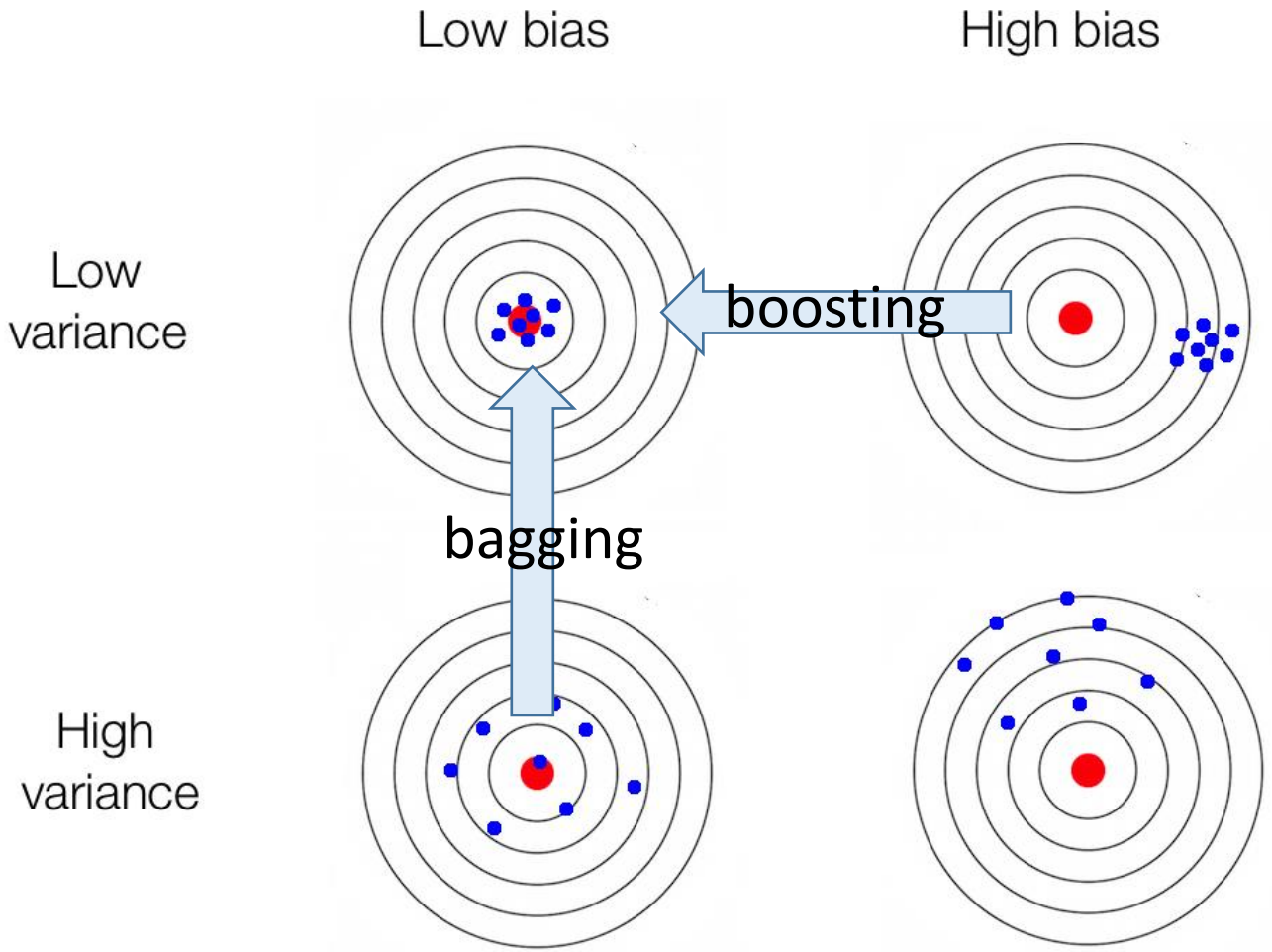
Error due to bias

Error due to variance

Irreducible error



Bagging and boosting use under- and overfitting to get more performant models



Bagging uses multiple overfitted (high variance) trees to reduce overfitting (low variance)

- Simulate the notion of different train set samples:

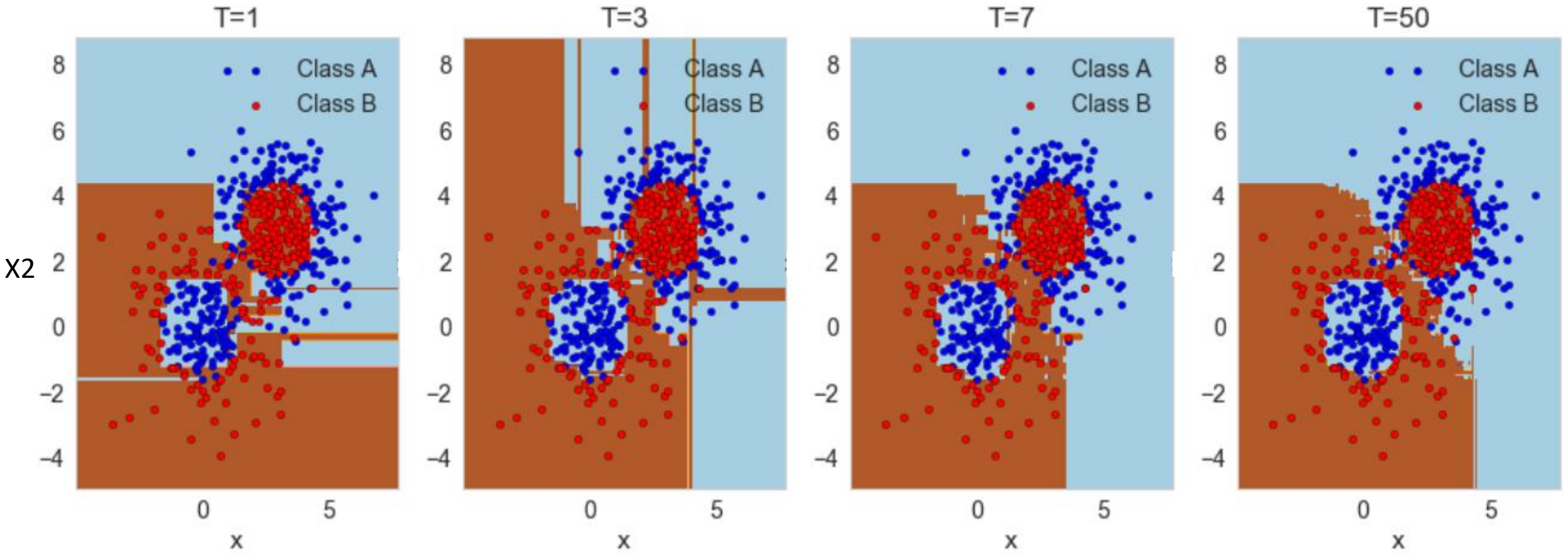
    1. sample data points from train set to create new train set
    2. fit low bias high variance model on new train set
    3. repeat steps (1) and (2) *T* times
    4. average the predictions of the *T* models

$$\hat{f}(x,\theta) = \frac{1}{T}\sum_{t=1}^{T} f_t(x,\theta)$$

Bagging: Random Forest

- Train set contains *n* data points with *m* features.
- Construct *T* low bias high variance decision trees by following these steps:
    1. Sample ***n*** data points at random **with replacement** from the train set.
    2. At each node, select *h* from *m* features at random where *(h<<m*) and compute the best split using only these *h* features.
    3. Each tree is grown to the largest extent possible. Or at least to a high-level complexity
- Step 3 ensures that the bagged models are low bias by learning deep complex decision trees.

Bagging: Random Forest



Boosting uses multiple underfitted (low variance; high bias) trees to reduce underfitting (low variance; low bias)

reduce the bias of a high bias low variance model

turning an ensemble of weak learners into a strong learner

the meta-model is additive, i.e. Adaboost:

$$\hat{f}(x,\theta) = \sum_{t=1}^{T} \alpha_t f_t(x,\theta)$$

Boosting: Gradient boosting

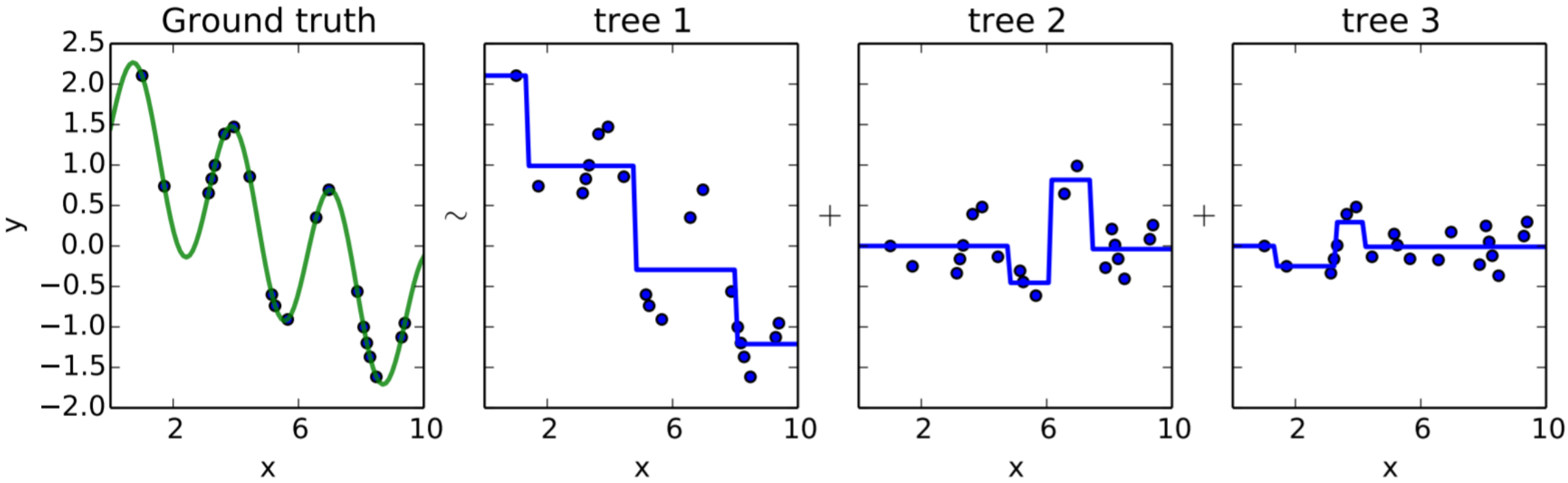
$$\hat{f}(x,\theta) = \sum_{t=1}^{T} f_t(x,\theta)$$

1. Fit a model $f_1(x,\theta) = y$
2. Fit a model to the **residuals** $h_1(x) = y - f_1(x,\theta)$
3. Create a new model $f_2(x,\theta) = f_1(x,\theta) + h_1(x)$

$$f_0(x,\theta) = \frac{1}{n}\sum_{i=1}^{n} y_i$$

$$f_{t+1}(x,\theta) = f_t(x,\theta) + h_t(x)$$

Boosting: Gradient boosting



[Gradient Boosting explained [demonstration]]